

TP1 – Logo et prise en main

Ces TP sont automatiquement évalués à partir d'une archive que vous devez déposer à la date et dans le dépôt prévus se trouvant sur la page Moodle de l'UE. Pour construire cette archive, vous devez taper la commande:

```
$ make tp1
```

*Cette commande produit le fichier **tp1-DATE.tgz** à déposer. Comme les TP se tiennent semaine après semaine, la date de dépôt est positionnée un jour avant la prochaine séance de TP. Pour chaque groupe de TP, un rappel des dates figure juste avant le dépôt. La date de dépôt n'est pas stricte mais vous ne serez évalué et vous ne pourrez avoir un retour sur votre travail que si vous déposez dans les temps.*

Attention : *l'usage de l'IA pour faire le TP est formellement interdit !*

Ce TP est principalement une introduction à l'environnement de développement de notre compilateur. Pour vous aider, un temps indicatif à passer sur le TP est donné pour chaque partie.

Le langage Logo (5 mn)

Logo est un langage de programmation à visée pédagogique conçu dans les années 70 et ayant connu un grand succès durant les années 80. Logo (*Logos* – en grec la parole, le discours, l'intelligence) a été inventé par [S. Papert](#) et W. Feurzeig en s'inspirant des concepts et idées pédagogiques de [Jean Piaget](#) sur le développement cognitif. Ce langage a pour objectif de mettre en œuvre la notion de [schème](#), qui est le résultat de l'adaptation de l'individu à son milieu, avec l'idée que conception et réalisation doivent être intimement liées pour réaliser l'apprentissage.

Un des modules, bien connu, de réalisation de concepts logiques est le module de la *Tortue Graphique*. Des commandes permettent de déplacer une tortue, représentant un point, à l'écran (voire des robots dans le monde réel) pour réaliser des dessins. En vérité, ce module aura un énorme succès qui se traduira par l'assimilation de Logo à sa *Tortue Graphique*.

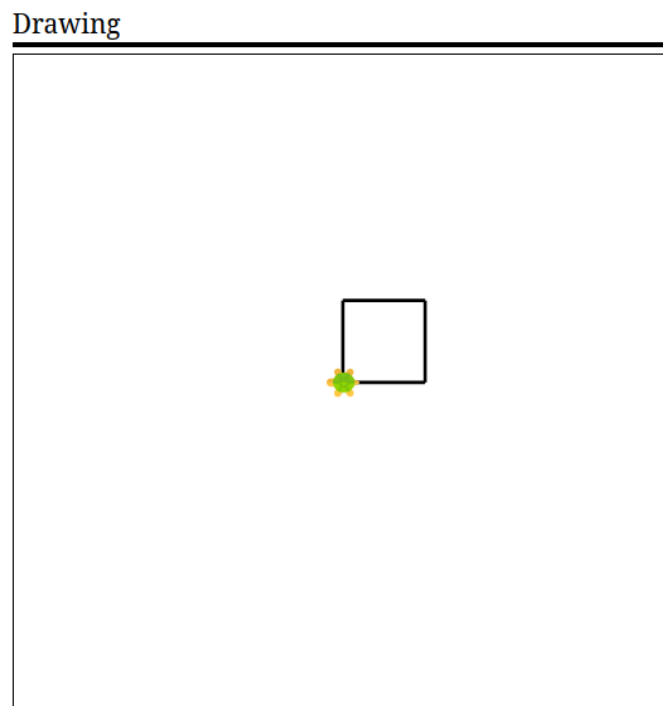
Un programme Logo est une suite de commandes nécessitant éventuellement 0 ou plusieurs paramètres.

```
forward 50
right 90
forward 50
right 90
forward 50
right 90
forward 50
```

Par exemple, le programme ci-dessus permet de tracer un carré :

- forward n – fait avancer le robot de n unités graphiques,
- right n – fait tourner le robot à droite de n degrés.

Ce qui donne, dans notre environnement, l’affichage suivant :



Pour plus d'informations sur Logo, vous pouvez consulter les sites suivants :

- [https://fr.wikipedia.org/wiki/Logo_\(langage\)](https://fr.wikipedia.org/wiki/Logo_(langage))
- <https://people.eecs.berkeley.edu/~bh/docs/html/usermanual.html>
- <http://www.cs.berkeley.edu/~bh/logo.html>

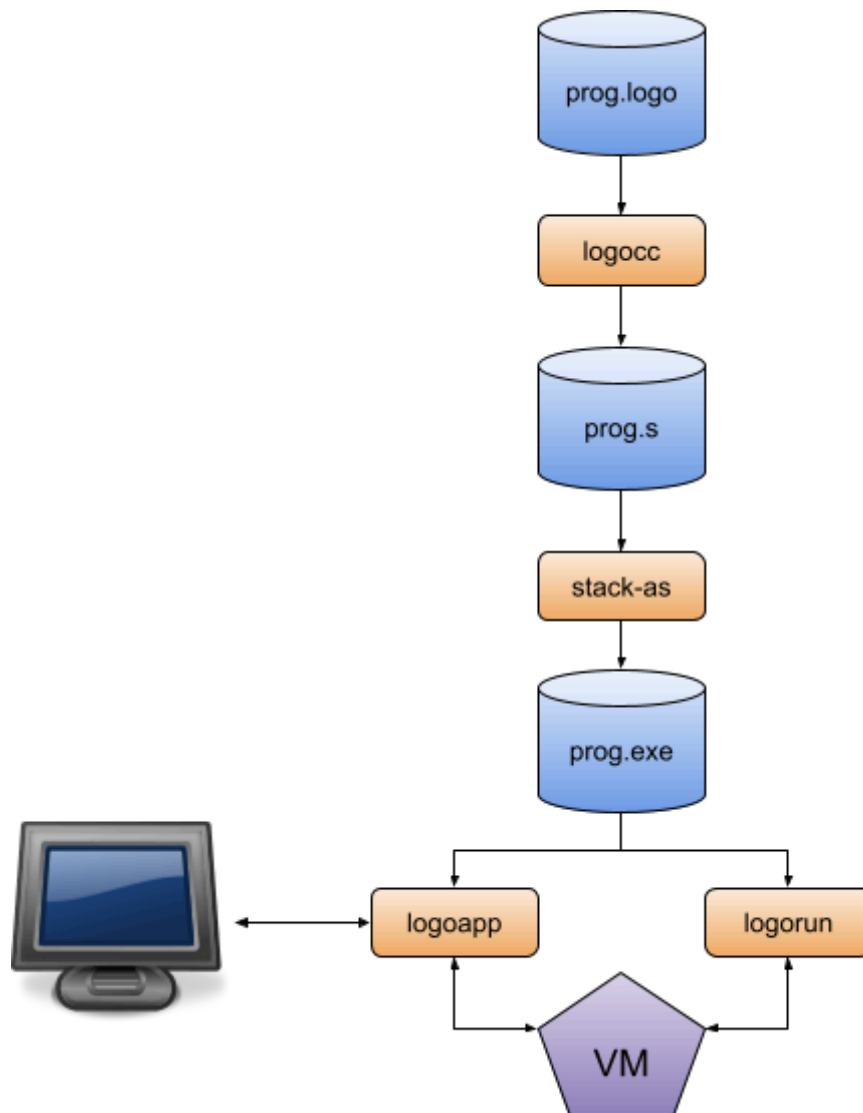
Parmi ses descendants, on peut compter le langage [Squeak](#), une implémentation de [SmallTalk](#) reprenant les concepts de Logo. A son tour, Squeak aura pour descendant [Scratch](#), un langage de programmation éducatif qui est aujourd’hui très populaire.

Environnement (15 mn)

Notre environnement Logo permet, tout en un, de réaliser un programme Logo, de le compiler, de l'exécuter et de le déboguer. On trouvera les composantes suivantes :

- une machine virtuelle (*virtual machine*, VM) à pile capable d'exécuter des programmes en s'interfaçant avec le module *Tortue Graphique*.
- `logocc` – le compilateur de Logo.
- `stack-as` – l'assembleur du langage machine de notre VM.
- `logoapp` – l'application graphique permettant d'exécuter et déboguer des programmes Logo.
- `logorun` – l'application textuelle permettant d'exécuter un programme Logo en mode texte.

Le schéma ci-dessous montre comment toutes ces composantes s'interconnectent entre elles :



À faire

1. Depuis la page Moodle, téléchargez l'archive contenant notre environnement Logo, logo-DATE.tgz, et décompactez dans votre répertoire préféré:

```
$ tar xvfz logo-DATE.tgz
```

2. Construisez l'environnement.

```
$ cd logo
```

```
$ make
```

3. Affichez notre premier programme.

```
$ cat progs/square.logo
```

4. Compilez le.

```
$ ./logocc progs/square.logo
```

5. Affichez sa traduction en assembleur (un assembleur spécifique à notre VM).

```
$ cat progs/square.s
```

6. Assemblez le code.

```
$ ./vm/stack-as progs/square.s
```

7. Exécutez le programme depuis notre environnement Logo.

```
$ ./logoapp progs/square.exe
```

La dernière command ouvre, dans votre navigateur préféré, une page avec l'interface de notre environnement Logo:

Logo V1.0 Logo Program Execution

Drawing



Console

Maching reset!

Program

```
0000: debug "main"
0001: push #50
0002: invoke 1
0003: push #90
0004: invoke 3
0005: push #50
0006: invoke 1
0007: push #90
```

PC SP FP

Stack

0
1
2
3
4
5
6

La fenêtre est composée de 4 parties:

- en haut à gauche – le tracé de la tortue (résultat du programme),
- en haut à droite – le programme en assembleur (la ligne bleue donne la prochaine instruction à exécuter),
- en bas à gauche – la console qui affiche différents messages dont des messages d'erreur,
- en bas à droite – la pile de la VM (une machine à pile).

Pour exécuter le programme, on clique sur **Run**. Pour remettre l'exécution à 0, on clique sur **Reset**. Pour exécuter en pas-à-pas (et ainsi visualiser l'état de la machine à pile exécutant le programme), on clique sur **Step** (un clic pour chaque pas d'exécution). Pour quitter l'application, on clique sur **Quit**.

Comme dans tout débogueur, il est possible de mettre des *Points d'arrêt* en cliquant sur la colonne à gauche des instructions. Les points d'arrêt servent seulement quand le programme est exécuté avec **Run** : l'exécution s'arrête dès qu'un point d'arrêt est rencontré.

A faire : jouer avec l'interface pour exécuter le programme `square.exe`, faire du pas à pas et observer la pile, mettre des points d'arrêt, etc.

La machine virtuelle (20 mn)

Ce TP utilise une machine virtuelle (VM) pour 2 raisons :

1. Le compilateur est portable et indépendant d'un matériel spécifique.
2. La mise en place du compilateur est plus facile car il n'y a pas besoin de prendre en compte les contraintes matérielles réelles comme le nombre limité de registres.

Le programme, dans la VM, est représenté par une séquence d'instructions placées à des adresses consécutives. Chaque instruction est composée d'une commande (aussi appelée *mnémonique*) suivie de 0 ou plusieurs arguments constants entiers.

Comme notre VM est une machine à pile, les valeurs des principaux arguments sont pris dans la pile. Par exemple, l'instruction ADD qui réalise une addition prend son premier argument X en le dépliant puis elle dépile son second argument Y et finalement empile le résultat $X + Y$.

Les instructions suivantes réalisent des calculs : on note $i: A \Rightarrow B$ l'effet de l'instruction i avec l'état de la pile A (avant exécution de l'instruction), produisant l'état de la pile B (après exécution de l'instruction). On note "::" pour concaténer les éléments dans la pile :

- **ADD:** $S::y::x \Rightarrow S::(x + y)$
- **SUB:** $S::y::x \Rightarrow S::(x - y)$
- **MUL:** $S::y::x \Rightarrow S::(x * y)$
- **DIV:** $S::y::x \Rightarrow S::(x / y)$
- **MOD:** $S::y::x \Rightarrow S::(x \% y)$ – reste de la division entière
- **POW:** $S::y::x \Rightarrow S::(x^y)$

On ajoute à cela les instructions spéciales suivantes :

- **PUSH #k:** $S \Rightarrow S::k$ – empilement de valeur constante
- **RESERVE k:** $S \Rightarrow S::0::\dots::0$ – réserve de la place pour k variables globales
- **GET_GLOB k:** $S \Rightarrow S::S[k]$ – empile la k -ième variable globale
- **SET_GLOB k:** $S::x \Rightarrow S ; S[k] \leftarrow x$ – modifie en place la k -ième variable globale

Afin de pouvoir stocker des valeurs de variable, il est nécessaire d'allouer de la place en mémoire (dans la pile). Si on a besoin de 4 variables, on réalise d'abord l'opération **RESERVE 4**. Chaque variable sera numérotée de 0 à 3, ce qui correspond à leur position par rapport au sommet de pile). Pour empiler la valeur de la variable 2, par exemple, on utilisera **GET_GLOB 2**. Pour modifier la valeur de la variable avec le valeur du sommet de pile, on utilisera **SET_GLOB 2**.

Pour gérer le flot de contrôle, comme dans un langage assembleur classique, on utilise des instructions de branchement conditionnelles ou non. L'adresse des instructions cibles d'un branchement est obtenue en préfixant celles-ci par une étiquette L : . On peut alors utiliser les instructions de branchement suivantes :

- **GOTO L:** $S \Rightarrow S$ – branche dans tous les cas à l'étiquette L .
- **GOTO_EQ L:** $S::y::x \Rightarrow S$ – branche si $x = y$.
- **GOTO_NE L:** $S::y::x \Rightarrow S$ – branche si $x \neq y$.
- **GOTO_LT L:** $S::y::x \Rightarrow S$ – branche si $x < y$.
- **GOTO_LE L:** $S::y::x \Rightarrow S$ – branche si $x \leq y$.
- **GOTO_GT L:** $S::y::x \Rightarrow S$ – branche si $x > y$.
- **GOTO_GE L:** $S::y::x \Rightarrow S$ – branche si $x \geq y$.

Finalement, le programme s'arrête dès qu'il trouve l'instruction **STOP**. Ces instructions peuvent être utilisées, par exemple, pour programmer des boucles, par

exemple, ci-dessous, une boucle réalisant trois itérations.

```
_start:
    reserve 1
    push #0
    set_glob 0
L1:
    push #3
    get_glob 0
    goto_ge L2
    push #1
    get_glob 0
    add
    set_glob 0
    goto L1
L2:
    STOP
```

Exercice 1 (fichier `code/ex1.s`) : saisissez le programme, assemblez le (voir les commandes dans la section précédente) et exécutez le pas à pas dans l'environnement graphique. Observez l'évolution de la pile à mesure que les instructions sont exécutées.

Les appels système (20 mn)

Pour envoyer des commandes au système graphique, le programme doit réaliser des appels systèmes avec l'instruction **INVOKE** *c* où *c* est le numéro de la commande invoquée. Selon la commande, des paramètres supplémentaires doivent être empilés. les commandes, leurs paramètres et leur effet sur la pile sont décrits ci-dessous :

- **cFORWARD** (1): $S::d \Rightarrow S$
déplacement en avant d'une distance *d* unités.
- **cBACKWARD** (2): $S::d \Rightarrow S$
déplacement en arrière d'une distance *d* unités.
- **cRIGHT** (3): $S::a \Rightarrow S$
rotation à droite de *a* degrés.
- **cLEFT** (4): $S::a \Rightarrow S$
rotation à gauche de *a* degrés.
- **cPRINT** (5): $S::x \Rightarrow S$
affiche l'entier *x*.
- **cPRINTS** (6): $S::i \Rightarrow S$
affiche la constante d'indice *i*.
- **cHOME** (7): $S \Rightarrow S$
ramène la tortue à la position initiale.

- **cRANDOM** (8): $S::k \Rightarrow S::\text{random}([0, k-1])$
génère un nombre aléatoire dans l'intervalle $[0, k-1]$.

Cette dernière instruction permet d'afficher des constantes ou méta-valeurs, qui peuvent être déclarées avec la syntaxe utilisée dans l'exemple suivant :

```
.meta          0    "Hello, World!"
_start:
  push         0
  invoke       6
  stop
```

Les méta-valeurs supportées peuvent être des booléens (true ou false), des entiers, des flottants, des chaînes de caractères, ou même des étiquettes.

Exercice 2 (fichier `code/ex2.s`) : écrivez un programme qui trace un triangle équilatéral de 40 unités de longueur de côté dont la base est horizontale. On notera, qu'au démarrage, la Tortue Graphique est placée au centre de la feuille et orientée vers le haut. Le point gauche de la base de ce triangle doit être le point initial du tracé et on commencera par tracer la base.



Exercices (60 mn)

Exercice 3 (fichier `code/ex3.s`) : écrivez un programme qui affiche les 10 premiers entiers, entre 0 et 9, ainsi que leur somme.

Exercice 4 (fichier `code/ex4.s`) : écrivez un programme qui affiche une ligne brisée horizontale constituée de 20 modèles composés d'un segment montant (45°) de dimension 10 unités et d'un segment descendant (45°) de dimension 10.



Exercice 5 (fichier `code/ex5.s`) : réalisez une boucle itérant 10 fois. A chaque itération, un nombre aléatoire dans l'intervalle $[0, 10]$ est tiré. Si le nombre est inférieur à 5, la tortue tourne à droite de 30° ; sinon elle tourne à gauche de 30° . Dans tous les cas, elle avance ensuite de 20 unités.